

Operators

Unlock more complex operators at Expert, Master and Grandmaster Genius levels.

Arithmetic

Operator 	Description 
<code>abs(x)</code> base	Returns the absolute value of a number, removing any negative sign. Show more
<code>add(x, y, filter = false), x + y</code> base	Adds two or more inputs element wise. Set filter=true to treat NaNs as 0 before summing. Show more
<code>densify(x)</code> base	Converts a grouping field of many buckets into lesser number of only available buckets so as to make working with grouping fields computationally efficient Show more
<code>divide(x, y), x / y</code> base	Returns x divided by y (x / y). Note: dividing by zero raises an error; to avoid it, use <code>divide(x, add(y, 0.0001))</code> ; adding a small epsilon to the denominator prevents divide-by-zero errors.
<code>inverse(x)</code> base	Returns the reciprocal of x ($1 / x$). Note: errors when $x = 0$; to avoid it, use <code>inverse(add(x, 0.0001))</code> ; adding a small epsilon prevents divide-by-zero errors.
<code>log(x)</code> base	Calculates the natural logarithm of the input value. Commonly used to transform data that has positive values. Show more
<code>max(x, y, ..)</code> base	Maximum value of all inputs. At least 2 inputs are required Show more
<code>min(x, y ..)</code> base	Minimum value of all inputs. At least 2 inputs are required Show more

power(x, y)
base

Returns x raised to the power of y (x^y). Note: power(x, y) can drop the sign of x when y is non-integer; use signed_power(x, y) to preserve the sign of x.

[Show more](#)

reverse(x)
base

- x

sign(x)
base

Returns the sign of a number: +1 for positive, -1 for negative, and 0 for zero. If the input is NaN, returns NaN. Input: Value of 7 instruments at day t: (2, -3, 5, 6, 3, NaN, -10) Output: (1, -1, 1, 1, 1, NaN, -1)

[Show more](#)

signed_power(x, y)
base

x raised to the power of y such that final result preserves sign of x

[Show more](#)

sqrt(x)
base

Returns the non-negative square root of x. Equivalent to power(x, 0.5). Note: for $x < 0$ the result is undefined; to retain the sign of x, use signed_power(x, 0.5) instead.

[Show more](#)

subtract(x, y,
filter=false), x - y
base

Subtracts inputs left to right: $x ? y ? \dots$. Supports two or more inputs. Set filter=true to treat NaNs as 0 before subtraction.

[Show more](#)

Logical

Operator 	Description 
and(input1, input2) base	Returns 1 ('true') if both inputs are 1 ('true'). Otherwise, returns 0 ('false').
if_else(input1, input2, input 3) base	The if_else operator returns one of two values based on a condition. If the condition is true, it returns the first value; if false, it returns the second value. Show more
input1 < input2 base	Returns 1 ('true') if input1 is a smaller than input2. Otherwise, returns 0 ('false').
input1 <= input2 base	Returns 1 ('true') if input1 is a smaller or the same as input2. Otherwise, returns 0 ('false').
input1 == input2 base	Returns 1 ('true') if input1 and input2 are the same. Otherwise, returns 0 ('false').

`input1 >= input2`

base

Returns 1 ('true') if input1 is a larger or the same as input2. Otherwise, returns 0 ('false').

`input1!= input2`

base

Returns 1 ('true') if input1 and input2 are different numbers. Otherwise, returns 0 ('false').

`is_nan(input)`

base

If (input == NaN) return 1 else return 0

[Show more](#)`not(x)`

base

Returns the logical negation of x. Returns 0 when x is 1 ('true') and 1 when x is 0 ('false').

`or(input1, input2)`

base

Returns 1 if either input is true (either input1 or input2 has a value of 1), otherwise it returns 0.

Time Series

Operator 	Description 
<code>days_from_last_change(x)</code> base	Calculates the number of days since the last change in the value of a given variable. Show more
<code>hump(x, hump = 0.01)</code> base	Limits amount and magnitude of changes in input (thus reducing turnover) Show more
<code>kth_element(x, d, k, ignore="NaN")</code> base	Returns the K-th value from a time series by looking back over a specified number of ('d') days, with the option to ignore certain values. Commonly used for backfilling missing data. Show more
<code>last_diff_value(x, d)</code> base	Returns the most recent value of x from the past d days that is different from the current value of x. Show more
<code>ts_arg_max(x, d)</code> base	Returns the number of days since the maximum value occurred in the last d days of a time series. If today's value is the maximum, returns 0; if it was yesterday, returns 1, and so on. Show more
<code>ts_arg_min(x, d)</code> base	Returns the number of days since the minimum value occurred in a time series over the past d days. If today's value is the minimum, returns 0; if it was yesterday, returns 1, and so on. Show more

`ts_backfill(x,lookback = d, k=1)`

base

Replaces missing (NaN) values in a time series with the most recent valid value from a specified lookback window, improving data coverage and reducing risk from missing data.

Show more

`ts_corr(x, y, d)`

base

Calculates the Pearson correlation between two variables, x and y, over the past d days, showing how closely they move together.

Show more

`ts_count_nans(x ,d)`

base

Counts the number of missing (NaN) values in a data series over a specified number of days.

Show more

`ts_covariance(y, x, d)`

base

Calculates the covariance between two time-series variables, y and x, over the past d days. Useful for measuring how two variables move together within a specified historical window.

Show more

`ts_decay_linear(x, d, dense = false)`

base

Applies a linear decay to time-series data over a set number of days, smoothing the data by averaging recent values and reducing the impact of older or missing data.

Show more

`ts_delay(x, d)`

base

Returns the value of a variable x from d days ago. Use this operator to access historical data points by specifying the desired time lag in days.

Show more

`ts_delta(x, d)`

base

Calculates the difference between a value and its delayed version over a specified period. Useful for measuring changes or momentum in time-series data.

Show more

`ts_mean(x, d)`

base

Calculates the simple average (mean) value of a variable x over the past d days.

Show more

`ts_product(x, d)`

base

Returns the product of the values of x over the past d days. Useful for calculating geometric means and compounding returns or growth rates.

Show more

`ts_quantile(x,d, driver="gaussian" )`

base

Calculates the `ts_rank` of the input and transforms it using the inverse cumulative distribution function (quantile function) of a specified probability distribution (default: Gaussian/normal). This helps to normalize or reshape the distribution of your data over a rolling window.

Show more

`ts_regression(y, x, d, lag = 0, rettype = 0)`
base

Returns various parameters related to regression function

[Show more](#)

`ts_scale(x, d, constant = 0)`
base

Scales a time series to a 0–1 range based on its minimum and maximum values over a specified period, with an optional constant shift.

[Show more](#)

`ts_std_dev(x, d)`
base

Calculates the standard deviation of a data series x over the past d days, measuring how much the values deviate from their mean during that period.

[Show more](#)

`ts_step(1)`
base

Returns a counter of days, incrementing by one each day.

[Show more](#)

`ts_sum(x, d)`
base

Sum values of x for the past d days.

`ts_zscore(x, d)`
base

Calculates the Z-score of a time series, showing how far today's value is from the recent average, measured in standard deviations. Useful for standardizing and comparing values over time.

[Show more](#)

Cross Sectional

Operator 	Description 
<code>normalize(x, useStd = false, limit = 0.0)</code> base	Centers a daily cross section by subtracting the market mean; optionally divide by the cross sectional standard deviation and clamp the result to [?limit, +limit]. NaNs are ignored in mean/std.
	Show more
<code>quantile(x, driver = gaussian, sigma = 1.0)</code> base	Ranks and shifts a vector of Alpha values, then applies a chosen statistical distribution (gaussian, cauchy, or uniform) to reduce outliers. The sigma parameter controls the scale of the output.
	Show more
<code>rank(x, rate=2)</code> base	Ranks the values of the input x among all instruments, returning numbers evenly spaced between 0.0 and 1.0. Useful for normalizing data and reducing the impact of outliers.
	Show more
<code>rank(x, rate=1)</code> base	Scales the input so that the sum of absolute values across all instruments equals a specified

winsorize(x, std=4)
base

Winsorize limits values in a data to within a specified number of standard deviations from the mean, reducing the impact of extreme outliers. Note: recommended std values range from 2 to 5: std = 2, 3, 4, 5 removes approximately 4.5%, 0.27%, 0.01%, and 0.0001% of extreme values, respectively (higher std removes fewer extremes).

[Show more](#)

zscore(x)
base

Z-score is a numerical measurement that describes a value's relationship to the mean of a group of values. Z-score is measured in terms of standard deviations from the mean

[Show more](#)

Vector

Operator 	Description 
vec_avg(x) base	Calculates the mean (average) of all elements in a vector field for each instrument and date, converting vector data to a single matrix value. Show more
vec_sum(x) base	Calculates the sum of all values in a vector field. Show more

Transformational

Operator 	Description 
bucket(rank(x), range="0, 1, 0.1", skipBoth=False, NaNGroup=False) or bucket(rank(x), buckets = "2,5,6,7,10", skipBoth=False, NaNGroup=False) base	The bucket operator creates custom groups by dividing data into buckets (ranges) based on ranked values of any data field. These buckets can then be used with group operators like group_neutralize, group_rank, group_zscore etc. Show more
trade_when(x, y, z) base	The trade_when operator changes Alpha values only when a specific condition is met, keeps previous values otherwise, and can close positions by assigning NaN under an exit condition. It is useful for reducing turnover and controlling when trades are executed. Show more

Group

Operator 	Description 
--	---

`alpha, std = 4.0)`

base

mean of all non-NAN values over the past `alpha` days. The winsorized mean is computed by trimming extreme values based on a specified standard deviation multiplier (`std`, default 4.0).

Show more

`group_mean(x, weight, group)`

base

Calculates the harmonic mean of a data field within each specified group.

Show more

`group_neutralize(x, group)`

base

Neutralizes Alpha values within each specified group by subtracting the group mean from each value. Groups can be industry, sector, country, or any custom grouping.

Show more

`group_rank(x, group)`

base

Ranks each element within its group based on the input field, assigning a value between 0.0 and 1.0. This helps compare items within the same group, such as stocks in the same industry.

Show more

`group_scale(x, group)`

base

Normalizes values within each group to a range between 0 and 1, making data comparable across different groups.

Show more

`group_zscore(x, group)`

base

Calculates the Z-score of each value within its group, showing how far each value is from the group mean in terms of standard deviations. Useful for comparing values relative to their group.

Show more